INSTITUTO POLITECNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA EN INGENIERÍA Y TECNOLOGÍAS AVANZADAS

Práctica #3	Teoría de la Información	8 de enero de 2026

Robert Roa Karla Guadalupe

INDICE

Contenido

IMPLEMENTACIÓN EN HARDWARE DE UN CODIFICADOR Y DECODIFICADOR DE CANAL 2
Objetivo: 2
Introducción 2
Desarrollo 2
Matriz generadora: 2
Circuito Eléctrico 4
Código Arduino 4
Circuito lógico 7
Tabla palabra dato y palabra código 8
Tabla síndrome vs patrón de error 8
Resultados de la simulación 9
Listado de asignación de terminales 9
Conclusiones 10

IMPLEMENTACIÓN EN HARDWARE DE UN CODIFICADOR Y DECODIFICADOR DE CANAL

Objetivo:

- Implementación en hardware de codificadores de canal (bloque) mediante dispositivos lógicos programables (CPLD's), GAL's u otros dispositivos electrónicos programables.

Introducción

En los sistemas de comunicación modernos, la integridad de la información es un aspecto crítico. Durante la transmisión de datos a través de un canal real, es inevitable la presencia de ruido e interferencias que pueden alterar los bits enviados, provocando errores en la recepción. Para mitigar este problema, se utiliza la Teoría de la Información y, específicamente, la Codificación de Canal.

El objetivo principal de esta práctica es la implementación física de un sistema de codificación de bloque lineal sistemático (CBS), capaz de detectar y corregir errores2. En particular, se desarrollará un sistema CBS (7,3), donde cada palabra de código consta de n=7 bits totales generados a partir de k=3 bits de información.

El sistema se implementa utilizando un microcontrolador Arduino Mega, el cual simula las tres etapas fundamentales del diagrama de comunicación: el Codificador (Coder), el Canal (con inyección manual de errores mediante interruptores) y el Decodificador (Decoder). A diferencia de una transmisión paralela simple, este proyecto pone énfasis en la transmisión serial de los datos, visualizando el flujo de bits bit a bit mediante indicadores luminosos, cumpliendo con los requisitos de temporización y visualización del síndrome solicitados. Además, se aborda el reto de corregir no solo errores simples, sino patrones de errores dobles específicos asignados (bits 1,2 y 4,6), demostrando la capacidad del algoritmo de decodificación por síndrome para recuperar la información original.

Desarrollo

Matriz generadora:

21	ROBERT ROA KARLA GUADALUPE	1° CBS (7,3)	(1,2) y (4,6)

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Codificador de canal CBS (7,3)

1. **Entrada de Datos:** Se ingresan manualmente los 3 bits de información $\boldsymbol{d}$ mediante un banco de interruptores conectados a las entradas digitales del Arduino.
2. **Procesamiento Matricial:** El microcontrolador lee estos valores y aplica la **Matriz Generadora (*G*)** asignada para el caso 1° CBS (7,3) para calcular los bits de paridad.
3. **Ecuaciones de Codificación:** La palabra código $\boldsymbol{c} = (\boldsymbol{c_1}, \dots, \boldsymbol{c_7})$ se obtiene aplicando las siguientes ecuaciones lógicas (Suma Módulo 2 o XOR):

$c_1 = d_1$
$c_2 = d_2$
$c_3 = d_3$
$c_4 = d_1 \oplus d_2$
$c_5 = d_1 \oplus d_3$
$c_6 = d_2 \oplus d_3$
$c_7 = d_1 \oplus d_2 \oplus d_3$

4. **Visualización Serial:** La palabra código generada se transmite de manera serial (bit a bit) visualizándose en un LED testigo antes de ser almacenada y mostrada en la barra de LEDs de salida, verificando así el proceso de codificación y transmisión temporal.

Canal con inserción de errores

5. **Simulación del Canal:** El canal ruidoso se implementa de manera manual mediante un segundo banco de interruptores (DIP Switch de 7 vías), asignando un interruptor a cada bit de la palabra código.
6. **Inyección de Ruido:** Cada interruptor activado invierte el estado lógico del bit correspondiente, simulando una alteración en el medio de transmisión**.**
7. **Vector Recibido:** El vector resultante $\boldsymbol{r}$ se obtiene mediante la operación lógica XOR:
$$r_i = c_i \oplus e_i$$
donde e_i representa el error introducido en el canal.
8. **Tipos de Error:** Este método permite simular errores simples y, específicamente para esta práctica, los **errores dobles** asignados en los pares de bits (1,2) y (4,6).

Decodificador y cálculo del síndrome

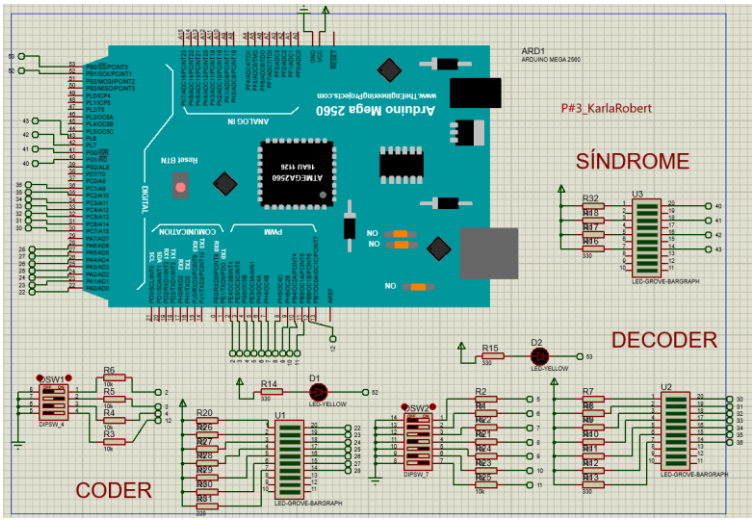
9. **Procesamiento:** El vector recibido $\boldsymbol{r}$ es capturado por la etapa decodificadora del Arduino.
10. **Cálculo del Síndrome:** Se calcula el vector de síndrome $s = (s_1, s_2, s_3, s_4)$ utilizando la **Matriz de Paridad (H)** mediante las ecuaciones:
$$s_1 = r_1 \oplus r_2 \oplus r_4$$
$$s_2 = r_1 \oplus r_3 \oplus r_5$$
$$s_3 = r_2 \oplus r_3 \oplus r_6$$
$$s_4 = r_1 \oplus r_2 \oplus r_3 \oplus r_7$$
11. **Diagnóstico Visual:** El síndrome calculado se muestra en 4 LEDs dedicados.

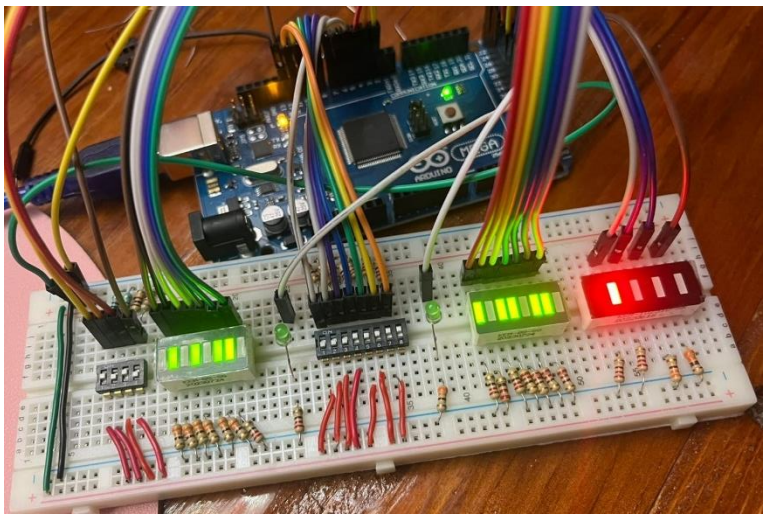
- ***Si s = 0000***, se asume que no hay error.
- ***Si s ≠ 0000***, el sistema detecta la presencia de una falla.

12. **Corrección Automática:** El algoritmo compara el síndrome obtenido con la tabla de patrones de error precalculada. Al identificar un síndrome conocido (como el decimal 6 para el error doble en bits 1 y 2, o el decimal 10 para el error en bits 4 y 6), invierte los bits afectados para restaurar el mensaje original.

13. **Salida Final:** Finalmente, la palabra corregida se muestra en la barra de LEDs de recepción, completando el ciclo de comunicación.

Circuito Eléctrico





Código Arduino

```
/*
 * PRACTICA 3: IMPLEMENTACIÓN DE CODIFICADOR DE CANAL
 * ALUMNA: Karla Guadalupe Robert Roa
 * ASIGNACIÓN: 1° CBS (7,3)
 * ---------------------------------------------------------
 * * Matriz Generadora G:
 * [ 1  0  0  1  1  0  1 ]
 * [ 0  1  0  1  0  1  1 ]
 * [ 0  0  1  0  1  1  1 ]
 */
```

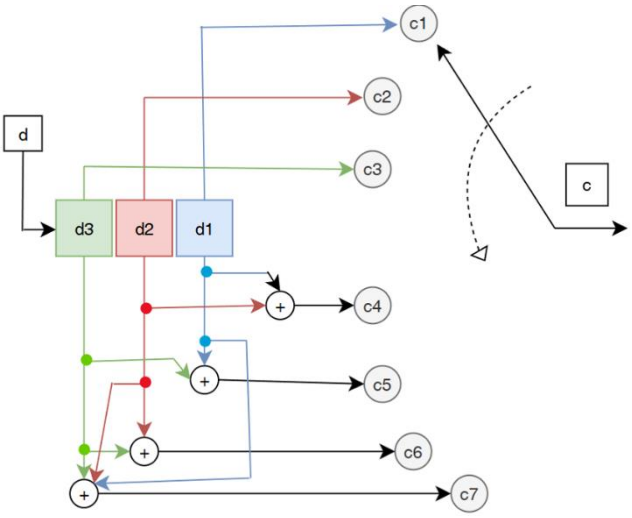
```
// --- AJUSTE DE LÓGICA DE LEDS ---
const int ENCENDIDO = LOW;
const int APAGADO   = HIGH;

// --- PINES ---
const int PIN_INICIO = 12;

// Entradas
const int PIN_DATO[3]   = {2, 3, 4};
```

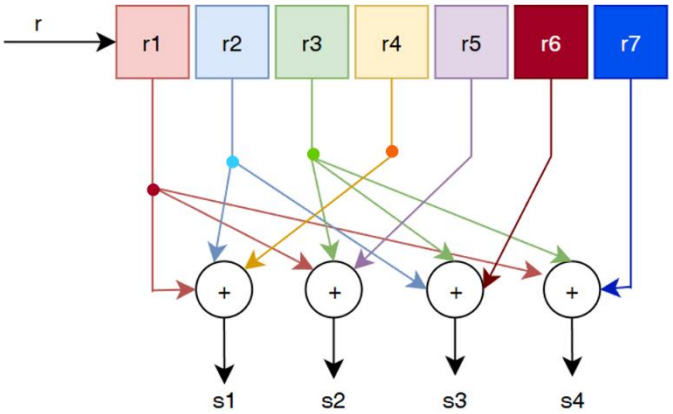
```
const int PIN_ERROR[7]  = {5, 6, 7, 8, 9, 10,
11};

// Salidas (Barras)
const int BARRA_CODIGO[7]   = {22, 23, 24,
25, 26, 27, 28};
const int BARRA_RECIBIDA[7] = {30, 31, 32,
33, 34, 35, 36};

// LEDs Seriales y Síndrome
const int LED_C_SERIAL = 52;
const int LED_R_SERIAL = 53;
const int LED_SINDROME[4] = {40, 41, 42, 43};

// Variables
int d[3]; int c[7]; int e[7]; int r[7]; int s[4];

// Matriz H (1° CBS 7,3)
const int H[4][7] = {
  {1, 1, 0, 1, 0, 0, 0},
  {1, 0, 1, 0, 1, 0, 0},
  {0, 1, 1, 0, 0, 1, 0},
  {1, 1, 1, 0, 0, 0, 1}
};

void limpiarLEDs() {
  for(int i=0; i<7; i++) {
    digitalWrite(BARRA_CODIGO[i],
APAGADO);
    digitalWrite(BARRA_RECIBIDA[i],
APAGADO);
  }
  for(int i=0; i<4; i++)
digitalWrite(LED_SINDROME[i], APAGADO);
  digitalWrite(LED_C_SERIAL, APAGADO);
  digitalWrite(LED_R_SERIAL, APAGADO);
}

void setup() {
  Serial.begin(9600);
  Serial.println("\n=== SISTEMA LISTO CON
CORRECCION VISUAL ===");
  Serial.println("Configura DATO y activa PIN 12
(5V) para iniciar.");

  pinMode(PIN_INICIO, INPUT);
  for(int i=0; i<3; i++) pinMode(PIN_DATO[i],
INPUT);
  for(int i=0; i<7; i++) pinMode(PIN_ERROR[i],
INPUT);
  for(int i=0; i<7; i++)
{ pinMode(BARRA_CODIGO[i], OUTPUT);
pinMode(BARRA_RECIBIDA[i], OUTPUT); }
  for(int i=0; i<4; i++)
pinMode(LED_SINDROME[i], OUTPUT);
  pinMode(LED_C_SERIAL, OUTPUT);
pinMode(LED_R_SERIAL, OUTPUT);

  limpiarLEDs();

  // Parpadeo inicial
  digitalWrite(LED_C_SERIAL, ENCENDIDO);
delay(500); digitalWrite(LED_C_SERIAL,
APAGADO);
}

void loop() {
  if (digitalRead(PIN_INICIO) == LOW) return;

  Serial.println("\n--------------------------------");
  Serial.println(">>> INICIANDO PROCESO...
<<<");
  limpiarLEDs();

  // --- 1. LEER DATO ---
  for(int i=0; i<3; i++) d[i] =
digitalRead(PIN_DATO[i]);

  Serial.print("1. Dato Entrada (d):    ");
  for(int i=0; i<3; i++) Serial.print(d[i]);
  Serial.println();

  // --- 2. CALCULAR CÓDIGO ---
  c[0] = d[0]; c[1] = d[1]; c[2] = d[2];
  c[3] = d[0] ^ d[1]; c[4] = d[0] ^ d[2]; c[5] = d[1]
^ d[2]; c[6] = d[0] ^ d[1] ^ d[2];

  Serial.print("2. Palabra Codigo (c):  ");
  for(int i=0; i<7; i++) Serial.print(c[i]);
  Serial.println("  <-- Enviando...");

  // ANIMACIÓN ENVÍO
  for(int i=0; i<7; i++) {
    int estado = (c[i] == 1) ? ENCENDIDO :
APAGADO;
    digitalWrite(LED_C_SERIAL, estado);
delay(300);
    digitalWrite(BARRA_CODIGO[i], estado);
    digitalWrite(LED_C_SERIAL, APAGADO);
delay(150);
  }
```

```
  Serial.println("   [PAUSA] Introduce ERRORES
ahora (5 seg)...");
  delay(5000);

  // --- 3. LEER ERRORES Y CALCULAR ---
  for(int i=0; i<7; i++) e[i] =
digitalRead(PIN_ERROR[i]);

  Serial.print("3. Vector Error (e):    ");
  for(int i=0; i<7; i++) Serial.print(e[i]);
  Serial.println("  <-- Leído del DIP");

  // Calcular Recibido CON ERROR (r = c ^ e)
  for(int i=0; i<7; i++) r[i] = c[i] ^ e[i];

  Serial.print("4. Palabra Recibida (r): ");
  for(int i=0; i<7; i++) Serial.print(r[i]);
  Serial.println("  <-- (Contiene Error)");

  // ANIMACIÓN RECEPCIÓN (Muestra lo que
llegó, CON error)
  for(int i=0; i<7; i++) {
    int estado = (r[i] == 1) ? ENCENDIDO :
APAGADO;
    digitalWrite(LED_R_SERIAL, estado);
delay(300);
    digitalWrite(BARRA_RECIBIDA[i], estado);
    digitalWrite(LED_R_SERIAL, APAGADO);
delay(150);
  }

  // --- 4. SÍNDROME Y DIAGNÓSTICO ---
  Serial.print("5. Sindrome:            ");
  for(int j=0; j<4; j++) {
    s[j] = 0;
    for(int i=0; i<7; i++) s[j] += r[i] * H[j][i];
    s[j] = s[j] % 2;

    digitalWrite(LED_SINDROME[j],
(s[j]==1)?ENCENDIDO:APAGADO);
    Serial.print(s[j]);
  }
  Serial.println();

  // Convertir Síndrome a Decimal
  int valS = (s[0]*8) + (s[1]*4) + (s[2]*2) +
(s[3]*1);

  //
=====================================
=========================
  // === LÓGICA DE CORRECCIÓN
(MODIFICA EL VECTOR r) ===
  //
=====================================
=========================

  if(valS == 0) {
    Serial.println(">> ESTATUS: Mensaje
CORRECTO (Sin Error)");
  }
  else {
    Serial.print(">> ESTATUS: ERROR
DETECTADO (S=");
    Serial.print(valS);
    Serial.println(") -> Corrigiendo...");

    // Aplicamos la corrección volteando los bits
afectados en 'r'
    // Usamos '!r[i]' para invertir el bit (si es 0 pasa
a 1, si es 1 pasa a 0)

    switch(valS) {
      // --- ERRORES SIMPLES (Por si acaso
pruebas uno solo) ---
      case 13: r[0] = !r[0]; break; // Error en bit 0
      case 11: r[1] = !r[1]; break; // Error en bit 1
      case 7:  r[2] = !r[2]; break; // Error en bit 2
      case 8:  r[3] = !r[3]; break; // Error en bit 3
      case 4:  r[4] = !r[4]; break; // Error en bit 4
      case 2:  r[5] = !r[5]; break; // Error en bit 5
      case 1:  r[6] = !r[6]; break; // Error en bit 6

      // --- TUS ERRORES DOBLES
ASIGNADOS ---
      case 6: // Error Doble (1,2) -> Bits 0 y 1
        Serial.println("   -> Corrigiendo par (1,2)");
        r[0] = !r[0];
        r[1] = !r[1];
        break;

      case 10: // Error Doble (4,6) -> Bits 3 y 5
        Serial.println("   -> Corrigiendo par (4,6)");
        r[3] = !r[3];
        r[5] = !r[5];
        break;

      default:
        Serial.println("   -> Error múltiple no
corregible.");
    }

    // === ACTUALIZACIÓN VISUAL DE LOS
LEDS ===
    // Ahora que 'r' ya está corregido, actualizamos
la barra de LEDs
```

```
  delay(1000); // Pequeña pausa dramática antes de corregir
  Serial.println(">> ACTUALIZANDO BARRA DE LEDS CON PALABRA CORREGIDA.");

  for(int i=0; i<7; i++) {
    int estado = (r[i] == 1) ? ENCENDIDO : APAGADO;
    digitalWrite(BARRA_RECIBIDA[i], estado);
  }
  }

  Serial.println(">>> FIN. Reinicia el PIN 12. <<<");

  while(digitalRead(PIN_INICIO) == HIGH) { delay(100); }
  delay(1000);
}
```

Circuito lógico





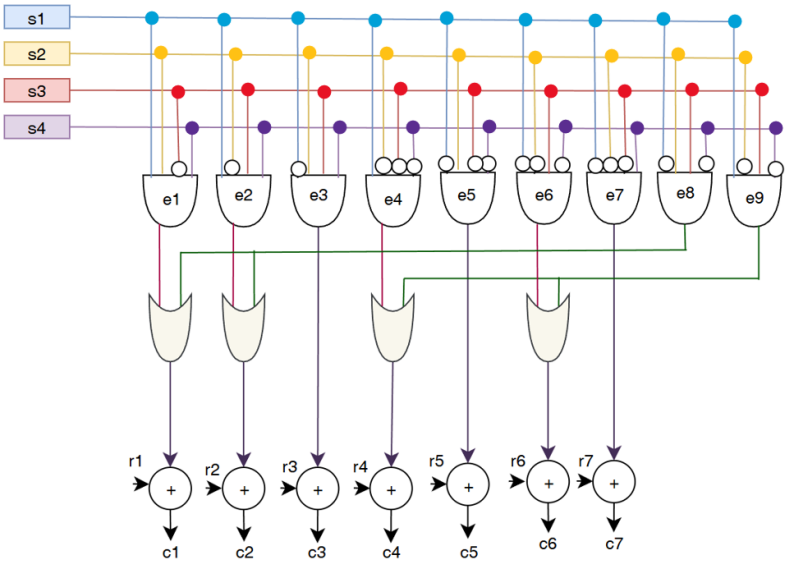


Tabla palabra dato y palabra código

d_1	d_2	d_3	c_1	c_2	c_3	c_4	c_5	c_6	c_7
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
0	**0**	**1**	**0**	**0**	**1**	**0**	**1**	**1**	**1**
0	**1**	**0**	**0**	**1**	**0**	**1**	**0**	**1**	**1**
0	**1**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	**0**
1	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**
1	**0**	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**1**
1	**1**	**0**	**1**	**1**	**0**	**0**	**1**	**1**	**0**
1	**1**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**1**

Esta tabla muestra las **8 combinaciones posibles** ($2^k = 2^3 = 8$) para los 3 bits de datos de entrada $\boldsymbol{d}$ y sus respectivas palabras código de salida $\boldsymbol{c}$ de 7 bits.
En ella se evidencia la estructura **sistemática** del código: los tres primeros bits c1-c3 copian exactamente el dato de entrada, mientras que los cuatro últimos c4-c7 corresponden a los bits de redundancia calculados por la Matriz Generadora G.

Tabla síndrome vs patrón de error

Descripción del Error	Patrón de Error e (e1 ,e2,e3,e4,e5,e6,e7)	Cálculo Matricial (Columnas de H)	Síndrome s (s1 ,s2,s3,s4)	Valor Decimal
Sin Error	0 0 0 0 0 0 0	-	**0 0 0 0**	**0**
Errores Simples				
Error en Bit 1 $\boldsymbol{d_1}$	1 0 0 0 0 0 0	Columna 1	**1 1 0 1**	**13**
Error en Bit 2 $\boldsymbol{d_2}$	0 1 0 0 0 0 0	Columna 2	**1 0 1 1**	**11**
Error en Bit 3 $\boldsymbol{d_3}$	0 0 1 0 0 0 0	Columna 3	**0 1 1 1**	**7**
Error en Bit 4 $\boldsymbol{p_1}$	0 0 0 1 0 0 0	Columna 4	**1 0 0 0**	**8**
Error en Bit 5 $\boldsymbol{p_2}$	0 0 0 0 1 0 0	Columna 5	**0 1 0 0**	**4**
Error en Bit 6 $\boldsymbol{p_3}$	0 0 0 0 0 1 0	Columna 6	**0 0 1 0**	**2**
Error en Bit 7 $\boldsymbol{p_4}$	0 0 0 0 0 0 1	Columna 7	**0 0 0 1**	**1**
Errores Dobles (Asignados)				
Par (1, 2)	1 1 0 0 0 0 0	Col 1 $\oplus$ Col 2	**0 1 1 0**	**6**
Par (4, 6)	0 0 0 1 0 1 0	Col 4 $\oplus$ Col 6	**1 0 1 0**	**10**

Esta tabla define la lógica de diagnóstico del decodificador. Relaciona cada **Síndrome** s, calculado como $s = r \cdot H^T$, con su patrón de error correspondiente.
Se detallan: el estado de transmisión limpia (**Síndrome 0**), los **7 errores de bit simple** (correspondientes a las columnas de la matriz H) y los **errores dobles asignados** (pares 1,2 y 4,6), mostrando los valores decimales únicos que permiten al sistema identificar y corregir cada caso.

Resultados de la simulación

Listado de asignación de terminales

La implementación se realizó utilizando un **Arduino Mega 2560**. A continuación, se detalla la correspondencia entre los pines físicos del microcontrolador y las variables lógicas del sistema CBS (7,3).

Entradas (DIP Switches y Control)

Pin Arduino	Dispositivo Físico	Variable Lógica	Descripción
D2	DIP Switch 1 - Pos 1	d_0	Bit de Dato 0 (LSB)
D3	DIP Switch 1 - Pos 2	d_1	Bit de Dato 1
D4	DIP Switch 1 - Pos 3	d_2	Bit de Dato 2 (MSB)
D5	DIP Switch 2 - Pos 1	e_0	Inyección Error en Bit 0
D6	DIP Switch 2 - Pos 2	e_1	Inyección Error en Bit 1
D7	DIP Switch 2 - Pos 3	e_2	Inyección Error en Bit 2
D8	DIP Switch 2 - Pos 4	e_3	Inyección Error en Bit 3
D9	DIP Switch 2 - Pos 5	e_4	Inyección Error en Bit 4
D10	DIP Switch 2 - Pos 6	e_5	Inyección Error en Bit 5
D11	DIP Switch 2 - Pos 7	e_6	Inyección Error en Bit 6
D12	Botón / Puente	Inicio	Gatillo de arranque (Active HIGH)

Salidas (Visualización y Diagnóstico)

Pin Arduino	Dispositivo Físico	Variable Lógica	Descripción
D22 - D28	Barra LEDs Verde	$\boldsymbol{c}$	**Palabra Código (Tx):** D28=c_0 ... D22=c_6
D30 - D36	Barra LEDs Verde	r	**Palabra Recibida (Rx):** D36=r_0 ... D30= r_6
D40	LED Individual	s_3	Bit de Síndrome 3 (LSB)
D41	LED Individual	s_2	Bit de Síndrome 2
D42	LED Individual	s_1	Bit de Síndrome 1
D43	LED Individual	s_0	Bit de Síndrome 0 (MSB)
D52	LED Amarillo	Serial TX	Testigo de tránsito (Coder →Canal)
D53	LED Amarillo	Serial RX	Testigo de tránsito (Canal → Decoder)

Porcentaje de Utilización del Dispositivo

De acuerdo con las especificaciones del fabricante, la placa **Arduino Mega 2560** cuenta con un total de **54 pines digitales** de entrada/salida (I/O).

Para la implementación del Codificador/Decodificador CBS (7,3), se utilizaron los siguientes recursos:

- **Total de Entradas:** 11 pines (3 Datos + 7 Errores + 1 Control).
- **Total de Salidas:** 20 pines (7 Código + 7 Recibida + 4 Síndrome + 2 Seriales).
- **Total de Pines Utilizados:** 31 pines.

$$Porcentaje\ de\ Utilización = \left(\frac{Pines\ Utilizados}{Total\ Pines\ Digitales}\right) x100$$

$$Utilización = \left(\frac{31}{54}\right) x100 \approx 57.4\%$$

El dispositivo opera a un 57.4% de su capacidad en cuanto a pines digitales, lo que indica que el Arduino Mega es una plataforma adecuada y sobrada para esta aplicación, permitiendo futuras expansiones o la adición de módulos de visualización LCD si fuera necesario, sin saturar los puertos del microcontrolador.

Conclusiones

El desarrollo integral de este proyecto permitió validar de manera experimental la robustez matemática de los Códigos de Bloque Lineales como mecanismo fundamental para garantizar la integridad de la información en canales de comunicación propensos a perturbaciones. A través de la implementación física y lógica en el microcontrolador, se demostró que el método del síndrome no se limita a una simple comprobación de paridad, sino que actúa como una herramienta determinista que, al integrarse con algoritmos de búsqueda optimizados, logra identificar y corregir con precisión tanto fallas individuales como los patrones de error doble específicamente asignados, superando las limitaciones de los métodos de detección convencionales. Adicionalmente, la simulación de la transmisión en tiempo real y la resolución de desafíos técnicos, como la adaptación a la lógica inversa del hardware, confirmaron que es factible reconstruir mensajes corruptos mediante la inversión selectiva de bits, evidenciando que la redundancia estructurada y una correcta abstracción entre software y hardware son pilares indispensables para asegurar la confiabilidad y la inmunidad al ruido en los sistemas de transmisión de datos modernos.